

Sistemas Basados en Microprocesador

Integración y desarrollo de
una aplicación:
Controlador de una válvula termostática

Alumno:

A: Sergio Fernandez Fernandez

B: David Calvo Heredero

Puesto Nº: 9

2025-26

Departamento de Ingeniería Telemática y Electrónica
Universidad Politécnica de Madrid

Índice del documento

1 OBJETIVOS DE LA PRÁCTICA	3
1.1 Resumen de los objetivos de la práctica realizada	3
1.2 Acrónimos utilizados	3
1.3 Tiempo empleado en la realización de la práctica.	4
1.4 Bibliografía utilizada	4
1.5 Autoevaluación.	5
2 RECURSOS UTILIZADOS DEL MICROCONTROLADOR	6
2.1 Diagrama de bloques hardware del sistema.	6
2.2 Cálculos realizados y justificación de la solución adoptada.	6
LCD	6
HORA	7
JOYSTICK	7
PC_COM	7
TEMP	8
PWM	8
3 SOFTWARE	9
3.1 Descripción de cada uno de los módulos del sistema	9
LCD	9
HORA	9
JOYSTICK	9
PC_COM	10
TEMP	10
PWM	10
3.2 Descripción global del funcionamiento de la aplicación. Descripción del autómata con el comportamiento del software (si procede)	11
LCD	11
HORA	11
JOYSTICK	12
PC_COM	12
TEMP	12
PWM	13
3.3 Descripción de las rutinas más significativas que ha implementado.	13
LCD	13
HORA	14
JOYSTICK	14
PC_COM	14
TEMP	14
PWM	15
4 DEPURACION Y TEST	16
4.1 Pruebas realizadas.	16
LCD	16
HORA	16
JOYSTICK	17
PC_COM	18

TEMP
PWM

18
19

Leyenda de formato

-  Verde → Extractos de código
-  Azul → Archivos .c/h
-  Negro → Destacar palabras/concepto/...

1 OBJETIVOS DE LA PRÁCTICA

1.1 Resumen de los objetivos de la práctica realizada

Se deben enumerar los objetivos que ha alcanzado al realizar la práctica. Enumérelos de forma precisa y sencilla.

Para esta primera parte del proyecto cumpliremos con los siguientes objetivos:

- ❖ **Puesta en marcha del RTOS:** Configuración del sistema operativo para ejecutar tres tareas a la vez: gestionar el Joystick, actualizar la pantalla LCD y contar la hora.
- ❖ **Generación de una señal PWM para controlar un buzzer:** Generación de una señal PWM de 1Khz de frecuencia y del 50 % de periodo activo. Con ello haremos funcionar un buzzer.
- ❖ **Control de un sensor de temperatura LM75B mediante un bus I2C:** Desarrollaremos el módulo de un sensor de temperatura manejado por un bus I2C.
- ❖ **Comunicación con el PC mediante protocolo UART:** Desarrollaremos un módulo para poder transmitir y recibir datos y que se muestren por la terminal del PC.
- ❖ **Control avanzado del Joystick:** Detección de las pulsaciones mediante interrupciones y diferenciar entre pulsación corta y pulsación larga usando temporizadores.
- ❖ **Manejo de la Pantalla LCD:** Desarrollo del módulo para escribir texto por pantalla y actualizar la imagen.
- ❖ **Comunicación entre Módulos:** desarrollo del Joystick mediante colas de mensajes, mostrando en el LCD qué botón se ha tocado y cómo (corto/largo).
- ❖ **Reloj Básico:** Implementación de la lógica de un reloj digital que cuenta segundos, minutos y horas en segundo plano.

1.2 Acrónimos utilizados

Identifique los acrónimos usados en su documento.

UART	Universal Asynchronous Receiver Transmitter
SPI	Serial Peripheral Interface
GPIO	General Purpose Input/Output
HAL	Hardware Abstraction Layer
CMSIS	Cortex Microcontroller Software Interface Standard
RTOS	Real-Time Operating System
LCD	Liquid Crystal Display
TIM	Timer
RCC	Reset and Clock Control

HSE	High Speed External (Oscillator)
PLL	Phase Locked Loop
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
MSB	Most Significant Bit
LSB	Least Significant Bit
CS	Chip Select
EXTI	External Interrupt
NVIC	Nested Vectored Interrupt Controller
IRQ	Interrupt Request
ISR	Interrupt Service Routine
CPOL	Clock Polarity
CPHA	Clock Phase
NMI	Non-Maskarable Interrupt
MOE	Main Output Enable
I2C	Inter-Integrated Circuit
PWM	Pulse Width Modulation
CCR	Capture/Compare Register
ARR	Auto-Reload Register
MOE	Main Output Enable
OC	Output Compare
AF	Alternate Function
SCL	Serial Clock
SDA	Serial Data

1.3 Tiempo empleado en la realización de la práctica.

Debe realizar una descripción sencilla del tiempo que ha dedicado a la realización de las actividades relacionadas con la práctica.



[Tiempo empleado para realizar la práctica]: El tiempo total empleado ha sido de 37 horas.

1.4 Bibliografía utilizada

- [RD1] [STM32F429-REFERENCE-MANUAL](#)
- [RD2] [Esquemático mbed-Application-Board](#)
- [RD3] [NUCLEO-F429ZI-USER MANUAL](#)
- [RD4] [STM32F429-DATASHEET](#)
- [RD5] [STM32F4 HAL Description](#)
- [RD6] [STLink Utility](#)

[RD7] Apuntes varios proporcionados por la universidad:
<https://moodle.upm.es/titulaciones/oficiales/course/view.php?id=773>

[RD8] IA, ej: ChatGPT, Gemini, Copilot...

1.5 Autoevaluación.

Autoevalúese comprobando los objetivos de aprendizaje indicados en la guía de la asignatura. Compruebe Si supera los objetivos de adquisición obligatoria. Si ha encontrado dificultades en la realización de la práctica indicelo.

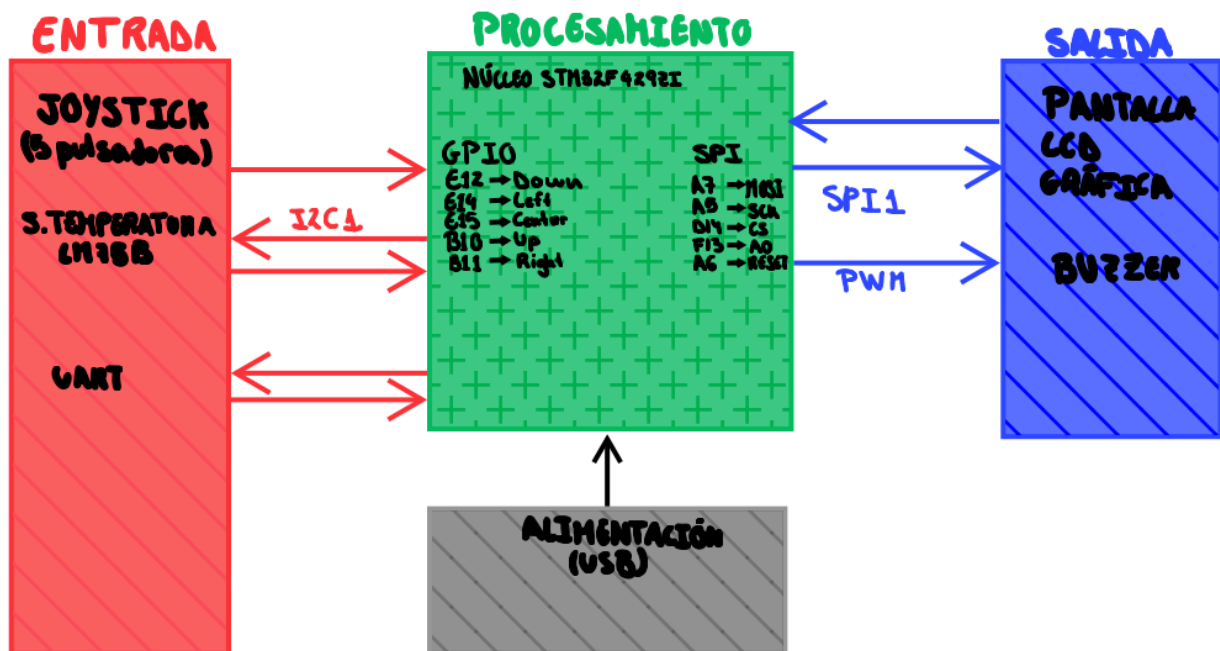
Durante el desarrollo del proyecto hemos cumplido con los siguientes objetivos de adquisición obligatoria:

- ❖ **Integración de Sistema Operativo en Tiempo Real:** Se ha cumplido con el resultado de aprendizaje, integrando la solución de la aplicación sobre un sistema operativo empujado. El código hace uso de CMSIS-RTOS2, implementando diferentes hilos sincronizados mediante colas de mensajes y flags, demostrando la capacidad de analizar arquitecturas software de mediana complejidad.
- ❖ **Manejo de Periféricos y Protocolos Estándar:** Se ha conectado y gestionado un periférico (pantalla LCD) utilizando interfaces basadas en protocolos estándar, concretamente SPI. Esto coincide con los contenidos del Bloque 2 impartidos en clase. Asimismo, se ha demostrado capacidad para manejar periféricos a partir de la documentación, configurando correctamente los registros y comandos de inicialización del display.
- ❖ **Gestión de Tiempos e Interrupciones:** Manejamos temporizadores hardware para gestionar la temporización precisa y la sincronización de la aplicación. Además, se han utilizado interrupciones externas para la gestión eficiente de eventos de entrada (como el Joystick), cubriendo los conocimientos exigidos en el Bloque 1 sobre arquitectura interna e interrupciones.

2 RECURSOS UTILIZADOS DEL MICROCONTROLADOR

2.1 Diagrama de bloques hardware del sistema.

Este apartado debe contener un diagrama de bloques donde se identifiquen claramente los elementos utilizados en el diseño o ejercicio. Debe elaborar una figura que muestre todos los elementos utilizados de la tarjeta NUCLEO STM32F429ZI y su interconexión con los elementos externos (tarjeta de aplicaciones, sensores, etc.).



2.2 Cálculos realizados y justificación de la solución adoptada.

En este punto debe describir cómo ha configurado cada uno de los recursos del microcontrolador, los cálculos que haya realizado y los valores programados en los registros más significativos.

Nose si se puede considerar como cálculos, pero esta es la lógica que hemos llevado a cabo para solucionar algunos problemas o requerimientos del sistema:

LCD

Algunos de los cálculos/lógica de este apartado nos los proporciona la universidad mediante algunos códigos como puede ser el de la función `symbolToLocalBuffer()`. Y en el resto se ha optado por medidas bastantes simples partiendo de esas funciones.

- ❖ **Uso de SPI vs GPIO:** Se ha optado por el periférico SPI hardware en lugar del GPIO por software debido al gran volumen de datos necesario para actualizar el **display (128x128)**. El SPI permite transferencias eficientes sin bloquear la CPU.
- ❖ **Buffer de Video Local:** Se implementa un **buffer** de memoria (`buffer[512]`) que actúa como memoria de vídeo local. Esto permite manipular los píxeles (escribir texto, borrar áreas) en la memoria del microcontrolador y plasmar el contenido al LCD, optimizando el rendimiento.

- ❖ **Temporización con TIM7:** Para asegurar los tiempos de espera del controlador del LCD (tales como los 10ms y 1ms en el reset), se configura el **TIM7** con un prescaler de 83 (para un reloj de 84MHz) para obtener una resolución de conteo exacta de 1 μ s.

HORA

Se ha optado por una implementación de reloj por software basada en un contador en lugar de utilizar un Timer hardware dedicado.

- ❖ **Base de Tiempos:** La función **osDelay(1000)** genera un bloqueo del hilo durante 1000 tics del sistema. Dado que la frecuencia del SysTick en la configuración estándar de CMSIS-RTOS suele ser de 1 kHz (1 ms por tick), este valor garantiza una periodicidad de **1 segundo** (1000x1ms=1s).
- ❖ **Arquitectura de Datos:** Se utilizan variables de tipo **uint8_t** ya que los valores de tiempo (0-59 y 0-23) caben perfectamente en 8 bits, optimizando el uso de memoria RAM.

JOYSTICK

Se ha optado a una comunicación orientada por eventos, mediante el uso de interrupciones externas y recursos del RTOS.

- ❖ **Gestión de Rebotes:** En vez de utilizar funciones del propio sistema como **delay**, se utiliza el **timer swtim** del RTOS. Esto permite validar la estabilidad de la señal sin detener la ejecución de otros hilos.
- ❖ **Tipos de pulsaciones:** Para diferenciar entre pulsación corta y larga, se utiliza el **timer timpuls**. Se inicia el conteo al detectar el flanco activo y cambia al soltar el botón o al pasarse el tiempo máximo (**timeout**).
- ❖ **Codificación de Gestos:** Se ha adoptado una codificación numérica para los gestos enviados a la cola, facilitando su decodificación en el módulo LCD: Arriba (1), Derecha (2), Abajo (4), Izquierda (8) y Centro (16). Esta implementación se podría haber hecho con diferentes números a estos pero decidimos hacerlos mediante potencias de 2 siguiendo así el **LSB** (Least Significant Bit).

PC_COM

Se implementa el protocolo de comunicacion UART.

- ❖ **Uso del protocolo UART:** Se ha optado por utilizar el **Driver_USART3** de la capa **CMSIS-Driver** en lugar de la **HAL** directa. Esto se hace para simplificar la comunicación entre pc y terminal.
- ❖ **Transmitir y Recibir datos:** Para no bloquear el sistema mientras se envían caracteres lentamente (**9600 baudios**), se ha implementado una **Cola de Mensajes (colaMensaje)**.
 - Cualquier hilo (**test.c**) inserta el mensaje en la cola instantáneamente.
 - El hilo **uart_msg** extrae y envía el dato cuando detecta el flag.
 - Importante decir que para recibir lo implementaremos más adelante
- ❖ **Sincronización por Flags:** Se utiliza **osThreadFlagsWait** y **osThreadFlagsSet** para **sincronizar** el envío. El hilo espera en estado bloqueado hasta que el hardware confirma con la interrupción

(**ARM_USART_EVENT_SEND_COMPLETE**) que el envío ha sido completado.

- ❖ **Configuración:** Se configura a **9600 Baudios, 8 bits de datos, sin paridad y 1 bit de parada..**

TEMP

Se utiliza el protocolo de comunicación serie síncrono I2C para la obtención de datos del sensor.

- ❖ **Uso de Driver I2C:** Al igual que en el puerto serie, se ha optado por utilizar la capa **CMSIS-Driver (Driver_I2C1)** para la gestión del bus.
- ❖ **Lectura y Conversión de Datos:** El sensor devuelve la temperatura en **dos bytes (MSB y LSB)**. Se ha implementado una lógica de desplazamiento de bits (**shifting**) para combinar ambos bytes en una variable de **16 bits**. Posteriormente, se aplica el factor de conversión (multiplicando por la resolución del sensor, típicamente 0.125 °C) para obtener el valor real en grados **Celsius**.
- ❖ **Periodicidad y Sincronización:** Para la actualización de 1 Hz, el hilo incluye un bloqueo temporal mediante **osDelay(1000)**. De esta forma, liberamos la CPU el resto del tiempo.
- ❖ **Comunicación entre Hilos:** Una vez procesado el dato numérico, este no se imprime directamente. Se envía a través de una **Cola de Mensajes (osMessageQueue)**.

PWM

- ❖ **Pin y Timer:** Se utiliza el pin **PE9** conectado al **TIM1_CH1**. Se selecciona este timer como si se seleccionara cualquier otro siempre que esté disponible, para que no haya problemas de concurrencia.
- ❖ **Cálculo del Tono de 1 kHz (Frecuencia Modificable a gusto propio):**
 - El reloj del sistema (**SystemCoreClock**) es de **168 MHz**.
 - El **Timer 1** se encuentra en el bus **APB2**. Dado que los divisores de reloj hacen que el timer reciba 168 MHz, hemos configurado un **Prescaler** de 167.
 - Cálculo: $168 / (167 + 1) = 1 \text{ MHz}$. Esto da una resolución de 1us.
 - Para obtener un **tono de 1 kHz**, configuramos el Periodo a 1000. ($1000 \text{ ticks} \times 1 \text{us} = 1 \text{ms} = 1 \text{kHz}$)
- ❖ **Activación MOE (Main Output Enable):** Al utilizar el **TIM1**, es obligatorio activar el bit **MOE** mediante la macro **__HAL_TIM_MOE_ENABLE**. A diferencia de algunos, si no se activa esta seguridad, la salida física permanece desactivada.

3 SOFTWARE

3.1 Descripción de cada uno de los módulos del sistema

En este punto debe explicar el funcionamiento de cada uno de los módulos que haya desarrollado.

LCD

El módulo del LCD ([lcd.c/lcd.h](#)) se encarga de recibir información de texto y representarla en el display.

- ❖ **Inicialización ([Init_Thread_LCD](#) y [Iniciacion_LCD](#)):** Configura los pines **GPIO**, inicializa el driver **SPI**, arranca el **TIM7** y realiza la secuencia de comandos de encendido del **display** (ajuste de contraste, dirección de barrido, etc.).
- ❖ **Hilo del LCD ([Thread_LCD](#)):** Es la tarea principal que permanece a la espera de datos. Funciona de manera **síncrona** con la cola de mensajes del sistema.
- ❖ **Driver Gráfico ([LCD_update](#), [escribe](#)):** Conjunto de funciones que traducen cadenas de caracteres **ASCII** a mapas de bits utilizando la fuente **Arial12x12** y gestionan el envío físico de los **bytes** a través del bus **SPI**.

HORA

El módulo de Hora ([hora.c/hora.h](#)) encapsula la lógica de cómo contar las horas/minutos/segundos.

- ❖ **Inicialización ([Init_Thread_hora](#)):** Esta función, llamada desde el **main**, se encarga de crear e iniciar el **hilo [Thread_hora](#)** dentro del sistema operativo .
- ❖ **Hilo de Tiempo ([Thread_hora](#)):** Es la tarea que mantiene en estado activo al **reloj**. Se ejecuta en un bucle infinito que alterna entre un estado de espera controlado por **osDelay** y un estado de ejecución.
- ❖ **Lógica de Conteo ([incrementarsegundo](#)):** Implementa un **contador ascendente**, gestionando los desbordamientos de segundos a minutos y de minutos a horas .

JOYSTICK

El módulo de Joystick ([joystick.c/joystick.h](#)) actúa como el controlador de entrada del sistema:

- ❖ **Inicialización ([Init_Joystick](#)):** Responsable de configurar los **periféricos** (**GPIOs**, **NVIC** para **interrupciones**) y de crear el **hilo**, **timers** y **colas** del sistema operativo.
- ❖ **Hilo del Joystick ([Thread](#)):** Es el cerebro del joystick. Permanece en estado de **wait**, esperando una señal. Al despertar, realiza la lectura física de los pines, aplica la lógica de **debounce** enviando los datos procesados a las colas de mensajes.
- ❖ **Manejador de Interrupciones ([stm32f4xx_it.c](#)):** Contiene la función **EXTI15_10_IRQHandler** y el callback **HAL_GPIO_EXTI_Callback**. Su única función es notificar al **hilo** mediante un **flag**, manteniendo el tiempo de interrupción al mínimo.

PC_COM

El módulo de comunicación ([pc_com.c/pc_com.h](#)) gestiona la transmisión de datos por el puerto serie.

- ❖ **Inicialización ([UART_Init](#)):** Crea la cola de mensajes, configura el **Driver_USART3** (velocidad y modo asíncrono) y lanza el hilo para la transmisión.
- ❖ **Hilo de Transmisión ([uart_msg](#)):** Es el que gestiona la cola. Espera **indefinidamente** hasta que llegue un mensaje y gestiona su envío al driver.
- ❖ **Interfaz Pública ([UART_MSG](#)):** Función auxiliar que permite a otros módulos ([test.c](#) o [main.c](#)) poner mensajes forma simple y sencilla.
- ❖ **Callback ([myUSART_callback](#)):** Función llamada desde la **ISR** del driver **UART** para “decir” al hilo que la transmisión física ha finalizado.

TEMP

El módulo de Temperatura ([Thread_Temp.c/Thread_Temp.h](#)) gestiona la recolección de datos del sensor LM75B a través del bus I2C.

- ❖ **Inicialización ([Init_Thread_temp](#), [init_I2C](#), [initCola](#)):** Se encarga de arrancar el hilo del sistema operativo, **configurar el Driver I2C** (estableciendo la velocidad **ARM_I2C_BUS_SPEED_FAST** y el control de energía) y crear la cola de mensajes (**colatemp**) donde se volcarán las medidas procesadas.
- ❖ **Hilo de Adquisición ([Threadtemp](#)):** Ejecuta un **bucle infinito** donde invoca la función de **lectura**, **envía** el dato resultante a la cola de mensajes global y despues se bloquea durante 1 segundo (**osDelay(1000)**) para liberar la CPU hasta la siguiente medida.
- ❖ **Lectura y Conversión ([read_Temp](#)):** Función que implementa la transacción **I2C**. Envía el comando de selección de registro, espera a que el bus esté libre y lee **dos bytes** de datos brutos. Posteriormente, realiza las operaciones de **desplazamiento** de bits (unir MSB y LSB y desplazar 5 posiciones) y aplica el factor de conversión (**0.125**) para obtener el valor final en grados Celsius.

PWM

El módulo del pwm ([ThreadPWM.c/ThreadPWM.h](#)) gestiona el buzzer.

- ❖ **Inicialización ([Init_TIM](#)):** Configura el pin **PE9** en modo alterno y establece los tiempos del **TIM1** para **PWM modo 1**.
- ❖ **Hilo de Control ([Thread](#)):** Programa la cadencia de los pitidos. **No genera la onda cuadrada** (eso lo hace el hardware), sino que enciende y apaga el **buzzer** modificando el **Duty Cycle**.
- ❖ **Arrancar hilo ([Init_ThreadPWM](#)):** Función llamada desde el [main.c](#) para iniciar el hilo y el timer.

3.2 Descripción global del funcionamiento de la aplicación. Descripción del autómata con el comportamiento del software (si procede)

En este punto debe explicar el funcionamiento completo de su aplicación. Debe aportar detalles de todos los elementos que forman la aplicación y de cómo interaccionan entre sí. Puede utilizar autómatas para describir el funcionamiento del software.

LCD

El funcionamiento del módulo LCD se basa en:

- I. **Estado de Espera:** El hilo **Thread_LCD** se encuentra bloqueado en la función **osMessageQueueGet**, esperando recibir un puntero a una cadena de caracteres.
- II. **Recepción de Datos:** Cuando otro hilo (**joystick.c** ó **test.c**(cualquiera)) deposita mensajes en **colamensaje**:
 - A. El LCD recibe el primer mensaje y lo interpreta como **Texto para la Línea 1**.
 - B. Inmediatamente espera y recibe el segundo mensaje como **Texto para la Línea 2**.
- III. **Escritura/Lectura en el Buffer Local:** Se llama a la función **escribe**. Esta función limpia los punteros de posición y utiliza **symbolToLocalBuffer_L1/L2** para buscar el patrón de bits de cada letra en el array **Arial12x12** y copiarlo al **buffer[512]** interno en la posición correcta.
- IV. **Actualización del display:** Una vez compuesto el **buffer**, se invoca a **LCD_update**. Esta función divide el buffer en **4 páginas (de 128 bytes cada una)** y las envía secuencialmente al controlador del LCD mediante la comunicación SPI, actualizando la imagen visible.
- V. **Retorno:** El hilo vuelve al estado de espera y cede la ejecución (**osThreadYield**).

HORA

El funcionamiento del módulo es cíclico y se basa en la precisión del RTOS:

- I. **Arranque:** El sistema operativo inicia el hilo **Thread_hora**.
- II. **Espera:** El hilo entra inmediatamente en la función **incrementarsegundo**, donde la primera instrucción es **osDelay(1000)**. El hilo cede la CPU y permanece en un estado de espera durante 1 segundo exacto.
- III. **Actualización:** Al despertar, se incrementa la variable **segundos**.
- IV. **Habilitación y reset:**
 - A. Si **segundos** alcanza 60: Se reinicia a 0 y se incrementa **minutos**.
 - B. Si **minutos** alcanza 60: Se reinicia a 0 y se incrementa **horas**.
- V. **Bucle:** El hilo termina la función, hace una llamada a **osThreadYield()** y vuelve a comenzar el bucle.

JOYSTICK

El funcionamiento se describe mediante el siguiente flujo de estados:

- I. **Estado de Espera:** El hilo del joystick está bloqueado y el hardware tiene las interrupciones habilitadas.
- II. **Evento de Interrupción:** Al accionar el joystick, se dispara la interrupción EXTI. La ISR envía un flag (0x01) al hilo.
- III. **Filtrado y Lectura:** El hilo despierta e inicia el timer. Tras el tiempo de guarda, lee el estado de los GPIOs para identificar la dirección del joystick.
- IV. **Notificación de Dirección:** Se envía inmediatamente el identificador de la dirección a la cola **colaJoystickgesto**. El módulo LCD consume este mensaje y muestra la dirección.
- V. **Medición de Duración:** Se inicia el timer de medición de pulsación. Al liberarse el botón, se comprueba el tiempo transcurrido:
 - A. Si tiempo < Umbral: Se envía evento de "Pulsación Corta" a **colaJoystickpulsacion**.
 - B. Si tiempo > Umbral: Se envía evento de "Pulsación Larga".
 - C. El módulo LCD actualiza la segunda línea del display con esta información.

PC_COM

El flujo de datos para la comunicación UART sigue los siguientes pasos:

- I. **Generación:** Un hilo (**Test_Thread**) llama a **UART_MSG("Texto")**. (en el proyecto final sera otro hilo quien lo llame)
- II. **Encolado:** El mensaje se mete en **colaMensaje**.
- III. **Procesado del dato:**
 - A. El hilo **uart_msg** (que estaba bloqueado esperando) despierta al recibir el dato de la cola.
 - B. Llama a **Driver_USART3.Send** para iniciar la transmisión.
 - C. El hilo se bloquea de nuevo esperando un **Flag (osThreadFlagsWait)**.
- IV. **Confirmación Hardware:** Cuando el UART termina de enviar el último bit del dato, se dispara el evento **ARM_USART_EVENT_SEND_COMPLETE**.
- V. **Notificación:** La función **myUSART_callback** establece el Flag (0x01), despertando al hilo **uart_msg** para que esté listo para el siguiente mensaje.

TEMP

El funcionamiento de este módulo combina la gestión de un protocolo de comunicación hardware (I2C) con la temporización del RTOS:

- I. **Configuración Inicial:** Al iniciar el hilo, se inicializa el **Driver I2C (init_I2C)**, configurando la velocidad del bus en modo rápido (**Fast Mode**) y activando la alimentación del periférico. También

se crea la cola de mensajes (**colatemp**) donde se depositarán los datos.

- II. **Solicitud de Medida:** Dentro de la función **read_Temp**, el **maestro** (STM32) inicia la comunicación enviando el comando de puntero (**0x00**) a la dirección del **esclavo** (**0x48**) mediante **MasterTransmit**. El sistema espera en un bucle a que el bus termine la transmisión.
- III. **Recepción de Bytes:** Inmediatamente después, se lanza una petición de lectura (**MasterReceive**) para obtener **2 bytes** del sensor. Nuevamente, se espera a que el hardware complete la recepción antes de continuar.
- IV. **Calculo:** Los dos bytes recibidos se **concatenan** y se desplazan 5 posiciones a la derecha para alinear los datos (**>>5**). El resultado se multiplica por el **factor de resolución** (**0.125**) para obtener la temperatura en grados **Celsius**.
- V. **Transmisión y Espera:** El valor calculado se envía a la cola **colatemp** mediante **osMessageQueuePut** para que otros módulos puedan usarlo. Finalmente, el hilo ejecuta **osDelay(1000)**, bloqueando su ejecución durante 1 segundo exacto antes de tomar la siguiente muestra.

PWM

El funcionamiento es un bucle infinito, basado en la modificación del registro de comparación (CCR1) dentro del bucle del hilo:

- I. **Estado ON:** El hilo pone el **Duty Cycle al 50%** usando **__HAL_TIM_SET_COMPARE**. Esto hace que el buzzer emita el tono de **1 kHz**.
- II. **Espera Activa:** El hilo duerme durante **200 ms** (**osDelay(200)**), manteniendo el pitido.
- III. **Estado OFF:** El hilo baja el **Duty Cycle al 0%**. El pin se queda a nivel bajo constante (cesa el pitido).
- IV. **Espera Silencio:** El hilo duerme durante **800 ms** (**osDelay(800)**).
- V. **Resultado:** Se obtiene 200ms de nivel alto (suena) y 800ms de nivel bajo (silencio).

3.3 Descripción de las rutinas más significativas que ha implementado.

En este punto debe enumerar y describir la funcionalidad de las rutinas más importantes que ha implementado.

LCD

- ❖ **Thread_LCD(void *argument):** (En **lcd.c**) Implementa el bucle infinito. Su lógica garantiza que siempre se actualicen ambas líneas de texto, esperando dos mensajes consecutivos antes de actualizar la pantalla.

- ❖ **LCD_update(void)**: Función que gestiona el buffer de la memoria. Envía los comandos necesarios para gestionar el buffer del microcontrolador a la estructura del controlador LCD.
- ❖ **delay(uint32_t n_microsegundos)**: Rutina de bajo nivel que programa y arranca el timer **TIM7 one-shot** y espera al flag de actualización (**TIM_FLAG_UPDATE**). Recaltar que esta función se implementó ya que con el delay propio de la librería HAL nos daba problemas.

HORA

- ❖ **incrementarsegundo(void)**: (En **hora.c**) Es la función núcleo del módulo. Contiene la llamada **osDelay(1000)** que define al reloj. A continuación, se implementa la lógica para manejar el formato de tiempo **hh:mm:ss**, modificando directamente las variables globales definidas en el **main**.
- ❖ **Init_Thread_hora(void)**: Crea el hilo utilizando la API de CMSIS-RTOS2 (**osThreadNew**), permitiendo que el módulo sea portable y fácil de instanciar desde el main (**principal.c**).

JOYSTICK

- ❖ **HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)**: (En **stm32f4xx_it.c**) Esta función es llamada cuando ocurre una interrupción en los pines configurados. Su implementación consiste en una única línea **osThreadFlagsSet(tid_Thread, 0x01)**, que manda un flag y habilita al hilo gestor para procesar el evento.
- ❖ **Thread (void *arg)**: (En **joystick.c**) Implementa la máquina de estados principal. Gestiona los **timers** del RTOS para los **rebotes** y la **duración**, lee el puerto de entrada y realiza las llamadas a **osMessageQueuePut** para la comunicación del resto de hilos (como el LCD).

PC_COM

- ❖ **uart_msg(void *argument)**: (En **pc_com.c**) Implementa el bucle de **transmisión**. La lógica que tiene hace que los mensajes se envíen **secuencialmente**, uno detrás de otro, evitando que se mezclen caracteres de diferentes mensajes gracias al uso de **osMessageQueueGet** y la espera de **flags**.
- ❖ **myUSART_callback(uint32_t event)**: (En **pc_com.c**) Es el punto de conexión entre el **hardware** y el **RTOS**. Al detectar el evento de transmisión completa, libera al hilo de **transmisión** mediante **osThreadFlagsSet(tid_uart, 0x01)**.

TEMP

- ❖ **Threadtemp(void argument)**: (En **Thread_Temp.c**) Implementa el hilo de temperatura. Su bucle infinito garantiza el **muestreo** de 1 segundo mediante la llamada a **osDelay(1000)**. Se encarga de **gestionar** la lectura del sensor y de meter el dato procesado en la cola de mensajes (**colatemp**).
- ❖ **read_Temp(void)**: (En **Thread_Temp.c**) Función que encapsula la transacción I2C completa. Utiliza las **primitivas** del CMSIS-Driver (**MasterTransmit y MasterReceive**) para obtener los datos crudos y realiza las operaciones lógicas de desplazamiento de bits (shifting) y multiplicación por el factor de conversión (0.125) para obtener la temperatura real.

- ❖ **init_I2C(void):** (En **Thread_Temp.c**) Realiza la configuración inicial del **periférico** utilizando la estructura **ARM_DRIVER_I2C**. Es fundamental para establecer la velocidad del bus (**ARM_I2C_BUS_SPEED_FAST**) y activar la alimentación del periférico antes de intentar cualquier comunicación.

PWM

- ❖ **Init_TIM(uint32_t periodo):** (En **ThreadPWM.c**) Realiza la configuración con la librería **HAL**. Es muy importante el uso de **HAL_TIMEx_PWMN_Start** y **__HAL_TIM_MOE_ENABLE** dado que el **TIM1** tiene salidas diferentes y protecciones varias que se deben habilitar manualmente.
- ❖ **Thread(void *argument):** (En **ThreadPWM.c**) Bucle que alterna el **Duty Cicle**(0% vs 50%) sincronizado con **osDelay**.

4 DEPURACION Y TEST

4.1 Pruebas realizadas.

Descripción del método de prueba utilizado para comprobar que las rutinas funcionan adecuadamente. Resultados de los tests. Indicación explícita de si el test es correcto o incorrecto. En este punto debe hacer especial hincapié en definir:

1. *Cuál es el objetivo de la prueba, indicando los módulos implicados*
2. *Cuál es el proyecto de Keil que permite realizar la prueba*
3. *Cuáles son las condiciones de entrada que permiten ejecutar la prueba*
4. *Cuáles son los resultados que se esperan y cuáles son los realmente obtenidos.*

LCD

Descripción del método de prueba: Se ha utilizado un hilo de prueba (**test.c**) que pone mensajes en la cola para verificar la visualización.

- ❖ **Objetivo de la prueba:** Verificar la correcta inicialización del SPI, el funcionamiento del buffer gráfico y la recepción de mensajes RTOS.

Resultados de los tests:

- ❖ **Prueba 1 (Inicialización):** Al arrancar el sistema.
 - Resultado esperado: La pantalla debe encenderse.
 - Resultado obtenido: La pantalla muestra un fondo limpio tras la ejecución de **LCD_init**. **(Correcto)**
- ❖ **Prueba 2 (Escritura de Texto Estático):** El módulo **test.c** envía los punteros **ms1 ("hola que tal")** y **ms2 ("muy bien")**.
 - Resultado esperado:
 - Línea 1 (superior): "hola que tal"
 - Línea 2 (inferior): "muy bien"
 - Resultado obtenido: Los textos aparecen en las posiciones correctas y con la tipografía Arial 12x12. **(Correcto)**
- ❖ **Prueba 3 (Refresco):**
 - Resultado: La pantalla mantiene la imagen estable sin parpadeos, ya que la escritura en el buffer .El uso del RTOS permite que el refresco no congele el sistema. **(Correcto)**

HORA

Descripción del método de prueba: Dado que este módulo manipula variables en memoria, la prueba principal se realiza mediante los “watches” (Watch Window) del entorno de depuración (Keil uVision).

- ❖ **Objetivo de la prueba:** Verificar la precisión del intervalo de 1 segundo y la corrección lógica del cambio de minuto y hora.

Resultados de los tests:

❖ Prueba 1 (Frecuencia de Conteo):

- Acción: Se observa la variable **segundos** en tiempo real.
- Resultado esperado: La variable debe incrementar su valor una vez por segundo.
- Resultado obtenido: El incremento es constante y acorde al tiempo real (comparado con cronómetro externo). **(Correcto)**

❖ Prueba 2 (Cambio de Minuto):

- Acción: Dejar pasar el tiempo y ver que pasa.
- Resultado esperado: Tras 60 segundos, **segundos** debe pasar a 0 y **minutos** debe incrementar en 1.
- Resultado obtenido: La transición 58 -> 59 -> 00 segundos y +1 en minutos se realiza correctamente. **(Correcto)**

❖ Prueba 3 (Cambio de Hora):

- Acción: Se inicializan **minutos** = 59 y **segundos** = 58.
- Resultado esperado: Tras 2 segundos, **horas** debe incrementar.
- Resultado obtenido: Transición correcta de 00:59:59 a 01:00:00. **(Correcto)**

JOYSTICK

Descripción del método de prueba: Se ha verificado el correcto funcionamiento del sistema mediante el LCD, se nos aconsejó en clase que mediante leds era más fácil. Pero aprovechando el código ya hecho de una práctica anterior nos dispusimos a probarlo con el LCD de manera más visual.

- ❖ **Objetivo de la prueba:** Validar la detección correcta de las 5 posiciones y la detección de pulsación. Módulos implicados: Joystick y LCD.

Resultados de los tests:

❖ Prueba 1 (Pulsaciones):

Se pulsaron secuencialmente las direcciones Arriba, Abajo, Izquierda, Derecha y Centro.

- Resultado esperado: El LCD debe actualizar la primera línea con el nombre de la dirección correspondiente inmediatamente.
- Resultado obtenido: El LCD mostró los textos correctos ("Arriba"...) sin error. **(Correcto)**

❖ Prueba 2 (Puls. Corta o Larga):

- Acción A: Pulsación breve (< 5s) en "Derecha".
 - Resultado: LCD muestra "Derecha" y luego "Puls. Corta". **(Correcto)**
- Acción B: Pulsación mantenida (> 5+-s) en "Derecha".
 - Resultado: LCD muestra "Derecha" y luego "Puls. Larga". **(Correcto)**

❖ Prueba 3 (Estabilidad):

Pulsaciones rápidas y repetitivas.

- Resultado: El sistema de filtrado funcionó adecuadamente. **(Correcto)**

PC_COM

Descripción del método de prueba: Se conecta la placa al PC. Abrimos el TeraTerm configurado a 9600 baudios en el puerto COM.

- ❖ **Objetivo de la prueba:** Verificar que la comunicación uart funciona, lo probaremos mediante el envío de cadenas de caracteres.

Resultados de los tests:

❖ Prueba 1 (Conexión Básica):

- Acción: Se ejecuta el sistema iniciando el hilo **Test_Thread** que envía el mensaje "Hola como estas??" cada segundo.
- Resultado esperado: Recibir en el Teraterm el texto programado repetidamente.
- Resultado obtenido: El terminal muestra la línea: **Hola como estas??**
Hola como estas??

.....

Cada segundo, de forma fluida y sin caracteres extraños. **(Correcto)**.

❖ Prueba 2 (Formateo de Datos - variable dinámica):

- Acción: Se ejecuta el sistema iniciando el hilo **Test_Thread** que envía el mensaje "El contador vale: x" cada segundo.
- Resultado esperado: Ver el número incrementándose.
- Resultado obtenido: Se recibía "El contador vale: 1", "El contador vale: 2"..... demostrando que **printf** funciona correctamente. **(Correcto)**.

TEMP

Descripción del método de prueba: Se ha utilizado el entorno de depuración de Keil (Watches) para monitorizar en tiempo real la variable **lectura** y la cola **colatemp**. Después, se aplicó estímulo físico (calor corporal) sobre el sensor.

- ❖ **Objetivo de la prueba:** Verificar la correcta comunicación I2C con el sensor LM75B, validar la conversión matemática de los datos (factor 0.125) y asegurar la periodicidad de muestreo establecida por el RTOS.

Resultados de los tests:

❖ Prueba 1 (Lectura en Reposo):

- Acción: Se inicia el sistema y se observa el valor devuelto por la función **read_Temp** sin interactuar con la placa.
- Resultado esperado: Obtener un valor estable y coherente con la temperatura ambiente del laboratorio (aprox. 22-25 °C).
- Resultado obtenido: La variable muestra un valor constante (ej: 23 °C) confirmando que la inicialización y la comunicación I2C son correctas. **(Correcto)**

❖ Prueba 2 (Variación Dinámica):

- Acción: Se coloca el dedo sobre el sensor de temperatura de la placa mbed durante unos segundos.

- Resultado esperado: El valor de la temperatura debe aumentar progresivamente y descender al retirar el dedo.
- Resultado obtenido: Se observa un incremento rápido de la variable (llegando a ~30 °C) y una recuperación lenta al valor ambiente, validando la lógica de conversión. **(Correcto)**

PWM

Descripción del método de prueba: Se conecta el buzzer en el pin **PE9**. Se verifica el funcionamiento mediante la escucha de este.

- ❖ **Objetivo de la prueba:** Comprobar que el Timer 1 genera una señal PWM de 1 kHz y que el RTOS gestiona el encendido (200ms) y apagado (800ms) correctamente.

Resultados de los tests:

❖ Prueba 1 (Frecuencia del Tono):

- Acción: Se escucha el sonido emitido por el buzzer al arrancar el hilo **ThreadPWM**.
- Resultado esperado: Se debe escuchar un tono de 1 kHz (agudo), correspondiente a un periodo de 1 ms configurado en el Timer (**Period = 1000, Prescaler = 167**).
- Resultado obtenido: El buzzer emite un pitido de frecuencia constante de 1 kHz **(Correcto)**

❖ Prueba 2 (Cadencia Temporal / Ritmo):

- Acción: Se observa la repetición del sonido a lo largo de varios ciclos.
- Resultado esperado: El sonido debe activarse durante (200ms) y permanecer en silencio el resto del segundo (800ms), repitiéndose infinitamente.
- Resultado obtenido: Se percibe sonido más o menos durante esos periodos de tiempo. **(Correcto)**